Learned so far

Everything from Days 1–3 on one page. Read it before each warm-up. Claude adds to it after every session.

Patterns

Pattern	Trigger in the problem	Key line	Card
Hash map lookup	"Have I seen this value before?", a pair that adds up to a target	`if target - x in seen:`	Card 02 Hash map lookup
Counting	"How many times", "the same count of each letter"	`count[ch] = count.get(ch, 0) + 1`	Card 03 Counting and grouping
Grouping	Group the items that share something, like the same letters	`groups["".join(sorted(w))].append(w)`	Card 03 Counting and grouping
Top k	"The k most frequent"	`Counter(nums).most_common(k)`	Card 03 Counting and grouping
Running total	Each item becomes itself plus everything before it	`nums[i] += nums[i - 1]`	Card 11 Prefix sums

Four rules that prevent most bugs

1. **Inside the brackets is the key; after `=` is the stored value.** `seen[x] = i` stores position `i` under the number `x`.
2. **`=` replaces, `.append` adds.** `groups[label] = w` loses the earlier words; `groups[label].append(w)` keeps them all.
3. **Parentheses build, brackets read.** `Counter(s)` builds a counter; `count[ch]` reads from one.

4. **A position isn't a value or a key.** `i` from `enumerate` is a position; a count is how many times. Ask which one you have before indexing.

Python tools

Tool	What it gives you	Example
`enumerate(nums)`	(position, value) pairs	`for i, x in enumerate(nums):`
`set()`	"seen or not"	`if x in seen:` then `seen.add(x)`
`d.get(key, 0)`	the stored value, or 0 if the key is missing	`count.get(ch, 0)`
`Counter(s)`	item → count in one line; a missing key counts as 0	`Counter("banana")["a"]` gives 3
`Counter(s).most_common(k)`	the k most frequent (item, count) pairs, most frequent first	`Counter("banana").most_common(2)` gives `[('a', 3), ('n', 2)]`
`sorted("tea")`	a **list** of the letters, in order	`['a', 'e', 't']`
`"".join(letters)`	one string glued from a list of letters	`"".join(['a', 'e', 't'])` gives `'aet'`
`defaultdict(list)`	a dict whose new keys start with an empty list	`groups[label].append(w)` never crashes
`d.values()`	the stored values, without their keys	`list(groups.values())`
`for a, b in pairs:`	each pair unpacked into two names	`for num, count in pairs:`

Outside LeetCode, add `from collections import Counter, defaultdict` at the top.

Big-O

Cost	Means	Example	Steps for 1,000,000 items
O(1)	the same work for any size	one dict or set lookup	1
O(n)	one pass	counting the letters	~1,000,000
O(n log n)	sorting	sort both words, then compare	~20,000,000
O(n^2)	a loop inside a loop	for each item, scan the others	~1,000,000,000,000

- Drop constants: two separate passes are O(2n), which is O(n).
- Keep different sizes apart: a note and a magazine give O(m + n); n words of k letters give O(n · k).
- **Count the loops you can't see:** `Counter(s)` is a full pass, `sorted(w)` costs k log k for a word of k letters, and `most_common` sorts the counts.
- Space is the number of different keys: letters are capped at 26, so O(1); numbers aren't, so O(n).

RESET

R read it out loud · **E** a tiny example · **S** a slow solution · **E** examine the triggers · **T** template, then trace. Full version: RESET.

In every session: type **blank** for the next RESET step, **hint** for the next hint, **stop** to end. When your mind goes blank at T, the skeleton with blanks gets you moving.

Problems so far

Problem	Pattern	The idea in one line
0001 Two Sum	Hash map lookup	For each x, check whether `target - x` is already in `seen` (value → index)
0217 Contains Duplicate	Hash map lookup	A set: return True the moment x is already in it
0242 Valid Anagram	Counting	Count both strings' letters and compare the counts
0383 Ransom Note	Counting	For each note letter, the magazine needs at least as many

Problem	Pattern	The idea in one line
0387 First Unique Character in a String	Counting	Count first, then return the first index whose count is 1
0049 Group Anagrams	Grouping	Label each word with its sorted letters; same label, same box
0347 Top K Frequent Elements	Top k	`most_common(k)`, then keep only the numbers from the pairs
0205 Isomorphic Strings	Pairing (unfinished)	Each letter of s always pairs with one letter of t, and no two share a partner: two dicts
1480 Running Sum of 1d Array	Running total	Add the previous item, which already holds the total so far

Live status, with Dataview on:

```
TABLE WITHOUT ID file.link AS Problem, last_result AS Result, familiarity AS
Familiar, next_review AS "Next review"
FROM "03 Problems"
WHERE attempts > 0
SORT next_review ASC
```

Mistakes to avoid

Dicts and counters

- Swapping key and value: rule 1.
- `count(ch, 0)` needs `.get`: `count.get(ch, 0)`. Without it, Python tries to call the dict and crashes.
- `Counter[s]` should be `Counter(s)`: rule 3.
- Looping over a Counter gives its keys (the letters), not counts or positions. Loop over the string with `enumerate(s)`, then read `count[ch]`.
- `Counter(s)[i]` with a position `i` always gives 0: a position isn't a key.
- `groups[label] = w` loses words: rule 2.

Lists and loops

- `pairs[count]` uses a count as a position, so Python says "index out of range".
- `List` with a capital L isn't defined: use `list`.
- `reverse=True` belongs to `sorted`, not to `join`.
- `in` on strings looks for letters side by side: `"ac" in "abc"` is False.
- "Appears only once" means a count of 1. A letter taken from the string never has a count of 0.

Code style

- `def f(string = str)` sets a default value. A type hint uses a colon: `def f(string: str)`.
- `if a == b: return True` followed by `return False` is just `return a == b`.
- Name things for what they hold: `top_pairs`, not `duplicates`.

Big-O

- A loop inside a loop is $O(n^2)$, not $O(n \log n)$.
- Hidden loops cost time: `Counter`, `sorted` and `most_common`.
- Letter counts take O(1) space; number counts take O(n).

Habits

- Triggers come from the problem, and your own examples often show them: "only one a" → counting.
- Trace before trusting an example: the running sum of [1, 2, 3] is [1, 3, 6], not [1, 3, 5].